

Project: Best Teacher – Find the Perfect Teacher for Grades 6–13

(7-Member Team Version)

Best Teacher is a web-based platform built with the MERN stack (MongoDB, Express.js, React, Node.js). It connects students (or parents) with qualified teachers for grades 6 to 13. Users can search for teachers by subject, grade, location, and availability, then book lessons, leave reviews, and manage their profiles. Teachers can create detailed profiles, set availability, and manage booking requests. An admin oversees the platform, verifies teachers, and moderates content. A new **Messaging System** enables direct communication between students and teachers after a booking is confirmed.

7 Core Components (One per Team Member)

Each component is a **full-stack module** (React UI + Express API + MongoDB models) with its own **CRUD operations** and **validations**. The components integrate via shared data and REST APIs.

Team Member Assignments (7 Members)

Member	Component
1	Authentication & User Profile
2	Teacher Management
3	Search & Filter (with Saved Searches)
4	Booking / Request System
5	Reviews & Ratings
6	Admin Dashboard
7	Messaging / Chat System

1 Authentication & User Profile

Primary Entity	User
----------------	------

CRUD Operations:

- **C (Create):** Register as a student or teacher.
- **R (Read):** View own profile details.
- **U (Update):** Edit profile (name, contact, profile picture, change password).
- **D (Delete):** Permanently delete account (with confirmation).

What Happens Here?

- **Registration:** User selects role (student/teacher), enters name, email, password, and optional details. Frontend validates email format, password strength (min 8 chars, one uppercase, one number, one special char) and password confirmation. On submit, backend checks email uniqueness, hashes password with bcrypt, and creates a User document. Returns JWT token.
- **Login:** (not CRUD but part of auth) – User provides credentials, backend verifies and issues JWT.
- **View Profile:** Authenticated user fetches their data from `/api/users/me`.
- **Update Profile:** User edits fields; frontend validates inputs (e.g., new password confirmation, valid phone number). Backend updates the user document.
- **Delete Account:** User confirms deletion; backend removes the user and may also delete related data (e.g., teacher profile, bookings, reviews) or anonymise them.

Validations:

- **Frontend:** React Hook Form + Yup – email regex, password strength meter, required fields.
- **Backend:** express-validator – check email uniqueness, password hash, sanitise inputs.
- **Database:** Mongoose schema – required: true, unique: true for email, minlength for password.

Integrates with: All other components (provides authenticated user context).

2 Teacher Management

Primary Entity	TeacherProfile (extends User with teacher-specific fields)
----------------	--

CRUD Operations:

- **C (Create):** After registering as a teacher, user creates their teaching profile.
- **R (Read):** Public view of teacher profiles (for students); private view for the teacher to see their own profile.
- **U (Update):** Teacher edits qualifications, subjects, hourly rate, bio, availability, profile picture.
- **D (Delete):** Teacher can delete their profile (soft delete or permanent).

What Happens Here?

- **Create Profile:** Teacher fills a detailed form: subjects taught (with grade levels), qualifications (degrees, certificates), hourly rate, bio, location (online/in-person), weekly availability schedule. Frontend validates required fields and data types (e.g., hourly rate positive number). Backend creates a TeacherProfile linked to the user.
- **View Profile (public):** Students see teacher's info, subjects, ratings, and reviews. Data fetched from `/api/teachers/:id`.
- **Update Profile:** Teacher edits any field; availability is submitted as a structured JSON (e.g., days and time slots). Backend validates that availability slots are logical (e.g., end time > start time).
- **Delete Profile:** Teacher deletes their profile; backend may also deactivate associated bookings (or cancel them).

Validations:

- **Frontend:** Subjects selected from predefined list; hourly rate > 0; bio max 1000 chars; availability times must be valid (e.g., no overlaps in teacher's own slots).
- **Backend:** Check that the teacher owns the profile; ensure subjects are from allowed list; validate availability format.
- **Database:** Mongoose schema with enum for subjects, min for rate, custom validator for availability.

Integrates with: Search & Filter (provides teacher data), Booking System (availability used for booking), Reviews (displays ratings).

3 Search & Filter (with Saved Searches)

Primary Entities	TeacherProfile, SavedSearch
------------------	-----------------------------

CRUD Operations:

- **C (Create):** Student can save a search query (e.g., “Maths, Grade 10, Colombo”) as a SavedSearch.
- **R (Read):** Perform search with filters and view results; list saved searches.
- **U (Update):** Modify a saved search (change filters).
- **D (Delete):** Remove a saved search.

What Happens Here?

- **Search:** On the search page, student enters criteria: subject, grade, location, max hourly rate, availability (day/time). Frontend builds a query string and calls `/api/teachers/search`. Backend performs a filtered query on `TeacherProfile` (e.g., `subjects.subject matches`, `hourlyRate <= maxRate`, `availability matches requested slots`). Returns paginated results.
- **Save Search:** Student clicks “Save this search”. Frontend sends the search criteria to `/api/savedsearches`. Backend stores it under the student’s user ID.
- **View Saved Searches:** Student can see a list of their saved searches and click to re-run them.
- **Update/Delete:** Student edits a saved search (e.g., change subject) or deletes it.

Validations:

- **Frontend:** Search inputs – grade must be 6–13, max rate positive number, date/time selection must be future.
- **Backend:** Sanitise query parameters to prevent injection; ensure saved search belongs to the student.
- **Database:** `SavedSearch` schema with `userId`, `criteria (object)`, `createdAt`.

Integrates with: Teacher Management (provides data to search), Dashboard (could show latest saved searches).

Booking / Request System

Primary Entity	Booking
----------------	---------

CRUD Operations:

- **C (Create):** Student sends a booking request to a teacher.
- **R (Read):** Both student and teacher view their bookings (upcoming, past, pending).
- **U (Update):** Teacher accepts/rejects booking; student cancels before confirmation; both parties can mark a lesson as completed (or system auto-completes after scheduled time).
- **D (Delete):** Booking can be cancelled (soft delete or status change).

What Happens Here?

- **Create Booking:** On teacher's profile, student selects a date/time from teacher's available slots, adds optional message, and submits. Frontend validates that the slot is still free (by checking against existing bookings). Backend creates a Booking with status pending.
- **View Bookings:** Student/teacher sees lists filtered by status (pending, confirmed, completed, cancelled). Click to view details.
- **Update Status:** Teacher can accept (status → confirmed) or reject (cancelled) a pending request. Student can cancel a pending booking (cancelled). After lesson time, teacher or student can mark as completed (or system auto-completes). Backend ensures only valid transitions.
- **Delete/Cancel:** Cancellation sets status to cancelled; actual record remains for history.

Validations:

- **Frontend:** Cannot book past dates; slot must be in teacher's availability; prevent double-booking (check on UI, but final check on backend).
- **Backend:** Verify that the requested time slot is within teacher's availability and not already booked (atomic check). Ensure user roles: only students can create bookings, only the involved teacher/student can update.
- **Database:** Booking schema with status enum, timestamps, references to teacherId and studentId.

Integrates with: Teacher Management (availability), Reviews (only completed bookings can be reviewed), Messaging (after booking confirmed, messaging is enabled).

5 Reviews & Ratings

Primary Entity	Review
----------------	--------

CRUD Operations:

- **C (Create):** After a lesson is marked completed, student can leave a review for the teacher.
- **R (Read):** View all reviews on a teacher's profile (public).
- **U (Update):** Student can edit their own review (within a time limit, e.g., 7 days).
- **D (Delete):** Student can delete their own review; admin can delete inappropriate reviews.

What Happens Here?

- **Create Review:** On the booking details page (after lesson completed), student rates teacher (1–5 stars) and writes a comment. Frontend validates rating required, comment max length (e.g., 500 chars). Backend checks that the booking belongs to that student and teacher and is completed, then creates a Review.
- **View Reviews:** Teacher profile page displays all reviews with average rating. Backend aggregates ratings.
- **Update Review:** Student edits rating/comment. Backend ensures they own the review and that it's within editable period.
- **Delete Review:** Student deletes; admin can delete any review (via admin dashboard). Backend removes review and updates teacher's average rating.

Validations:

- **Frontend:** Rating 1–5, comment required (min 10 chars), profanity filter (optional).
- **Backend:** Verify booking completion, ensure review not duplicate, check ownership, sanitise comment.
- **Database:** Review schema with rating (1–5), comment, references to bookingId, teacherId, studentId, unique index on bookingId (one review per booking).

Integrates with: Booking System (ensures only completed bookings can be reviewed), Teacher Management (displays rating).

6 Admin Dashboard

Primary Entities	User, TeacherProfile, Booking, Review, Report
------------------	---

CRUD Operations: (Admin-level actions)

- **C (Create):** Admin can manually create user accounts, send notifications, add system-wide announcements.
- **R (Read):** View all users, teachers, bookings, reviews, reports with filtering and stats.
- **U (Update):** Verify teacher profiles (mark as verified), deactivate/activate users, moderate reviews (hide/approve), resolve reports.
- **D (Delete):** Delete any inappropriate content (reviews, teacher profiles, user accounts) with proper confirmation.

What Happens Here?

- **Dashboard Overview:** Admin sees key metrics (total users, teachers, bookings, pending verifications) via charts and cards. Data fetched from aggregated endpoints.
- **Manage Users:** List all users with role, status. Admin can search, filter, click to view details. Actions: deactivate, delete, change role.
- **Verify Teachers:** Pending teacher profiles (with uploaded certificates) appear. Admin can view documents, then approve (set verified: true) or reject with reason.
- **Moderate Reviews:** Flagged reviews or reports appear. Admin can view, delete, or dismiss reports.
- **Handle Reports:** Users can report a teacher or review. Admin reviews and takes action (warn, disable, etc.). Reports have CRUD (create by user, read by admin, update status, delete after resolution).

Validations:

- **Frontend:** Admin actions require confirmation modals; sensitive actions (delete) double-checked.
- **Backend:** Strict role-based access control (only users with role: admin can access these endpoints). All updates check for existence and permissions.
- **Database:** No special validations beyond standard; audit logs may be kept.

Integrates with: All components – it consumes data from every other entity.

7 Messaging / Chat System

Primary Entities	Conversation, Message
------------------	-----------------------

CRUD Operations:

- **C (Create):** Send a new message (implicitly creates a conversation if it doesn't exist).
- **R (Read):** List all conversations for the logged-in user; view messages inside a conversation (paginated).
- **U (Update):** Mark messages as read; optionally allow editing a message within a short time window.
- **D (Delete):** User can delete a message (soft delete for themselves) or admin can delete inappropriate messages globally.

What Happens Here?

- **Create Conversation & Message:** When a student (with a confirmed booking) navigates to the messaging section and sends a message to a teacher, the system checks if a conversation between the two already exists. If not, it creates a new Conversation document with the two participants. Then a Message document is created with sender ID, content, timestamp, and read: false.
- **List Conversations:** Fetch all conversations where the user is a participant, sorted by last message time. For each, include the last message preview and unread count.
- **View Messages:** Fetch messages for a specific conversation (paginated, newest first). Mark all messages sent to the user as read when the conversation is opened.
- **Update Message:** (Optional) Allow a user to edit their own message within 5 minutes of sending. Backend checks ownership and time window, then updates content.
- **Delete Message:** Users can delete their own messages (for themselves only – i.e., hide) or request admin to delete inappropriate messages. Admin can delete any message permanently.

Validations:

- **Frontend:** Message content cannot be empty; max length 1000 chars; no HTML/script injection.

- **Backend:**
 - Verify that the sender and recipient are both registered users.
 - Enforce messaging permission: only allow if they have at least one confirmed or completed booking together (prevents spam). This check is done when sending the first message; subsequent messages allowed as long as conversation exists.
 - For updates/deletes, ensure user owns the message.
- **Database:**
 - Conversation schema: participants [userId, userId], lastMessageAt, createdAt. Index on participants for quick lookup.
 - Message schema: conversationId, senderId, content, read (boolean), createdAt, updatedAt. Optionally deletedFor array to implement per-user deletion.

Integrates with: Booking System (to verify permission), User Profile (to display names/avatars), Admin Dashboard (to moderate messages).

Summary: CRUD Coverage Across 7 Components

Component	Primary Entity	Create	Read	Update	Delete
1. Auth & Profile	User	✓	✓	✓	✓
2. Teacher Management	TeacherProfile	✓	✓	✓	✓
3. Search & Saved Searches	SavedSearch	✓	✓	✓	✓
4. Booking System	Booking	✓	✓	✓	✓
5. Reviews & Ratings	Review	✓	✓	✓	✓
6. Admin Dashboard	(multiple)	✓	✓	✓	✓
7. Messaging	Conversation/Message	✓	✓	✓	✓

Integration Overview

- All components share the same MongoDB database.
- Authentication middleware (JWT) protects endpoints and provides req.user.
- React frontend uses global state (Context/Redux) to store authenticated user and shared data.
- API contracts are agreed upon early (e.g., using Swagger/OpenAPI) to ensure smooth integration.
- Events: When a booking is confirmed, messaging permission is implicitly granted; when a review is added, teacher's average rating is recalculated.

Alignment with IT2150 Learning Outcomes

LO	How Best Teacher Addresses It
LO1 (Plan & execute)	Use Agile (sprints), create WBS, assign components, risk register, Gantt chart.
LO2 (QA & Communication)	Code reviews, unit tests, daily stand-ups, meeting minutes, QA checklist.
LO3 (System development)	Secure auth (JWT, bcrypt), role-based access, clean UI (Material-UI), input validation (frontend/backend/db).
LO4 (Monitor progress)	Trello board, burndown charts, weekly progress reports.
LO5 (Collaborative team)	Pair programming, inclusive design (accessibility), equitable workload, respect diverse perspectives.
